AI System Architecture
&
Industrial AI Portfolio

Yen-Te Chou
Industrial AI & Robotics Engineer
AI-Assisted Engineering | System Integration | Computer Vision

Portfolio Overview

- **Part 0 — Who I Am**
  - Industrial AI System Architect mindset
  - End-to-end solution ownership
  - AI-assisted engineering execution
- **Part 1 — Engineering Philosophy**
  - Human-in-the-loop AI engineering
  - System-level architecture thinking
  - Rapid validation & iteration loop
- **Part 2 — Project A - AI Thermal Vision Pipeline**
  - Problem definition
  - System architecture
  - Model evolution
  - Deployment & impact
- **Part 3 — Project B - Industrial Automation Data Integration**
  - Manufacturing workflow issue
  - Python + SECS + RPA solution
  - System integration approach
  - Decision impact / efficiency gain

Part 4 — Project C - AI Safety Detection Concept

- Dataset & labeling challenge
- YOLO training validation
- Real-world constraint handling
- System deployment concept

Part 5 — Future Vision

- Central AI control center
- Edge AI architecture roadmap

Part 0 - Who I Am — Engineering Role & Value

- System-oriented industrial AI engineer focused on delivering real-world solutions rather than isolated technical components.
- Own the end-to-end lifecycle of engineering problems — from problem framing and architecture design to implementation, validation, and deployment.
- Leverage AI-assisted engineering workflows to accelerate prototyping while maintaining full responsibility for system decisions, validation strategy, and solution robustness.

Part 1 - How I Approach Industrial AI Problems

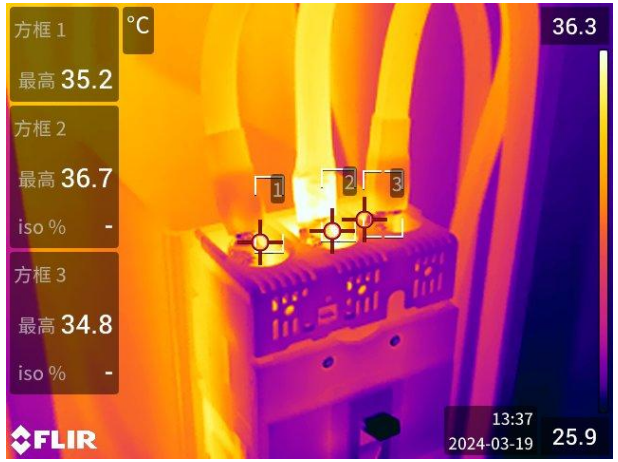
- Apply a human-in-the-loop AI engineering approach, combining domain expertise, system thinking, and AI-assisted implementation to accelerate solution development.
- Think in terms of system-level architecture rather than isolated components, ensuring scalability, maintainability, and deployment feasibility.
- Emphasize rapid validation and iteration loops, using data feedback, prototype testing, and incremental improvements to reduce engineering risk.

Part 2 - Manual Thermal Data Extraction Limiting Operational Efficiency

- Large volumes of thermal images were generated during equipment monitoring, requiring manual inspection and temperature recording.
- The process was time-consuming, error-prone, and lacked scalability as production data continued to increase.
- There was no automated mechanism to transform image-based thermal information into structured data for engineering analysis and decision making.
- A system-level solution was required to enable automated extraction, validation, and integration of thermal data into existing workflows.

2.1 - Problem

- Large volumes of thermal images required manual inspection and temperature recording.
- The process was slow, error-prone, and difficult to scale with increasing production data.
- Lack of structured data integration limited engineering analysis and decision-making efficiency.
- A system-level automation solution was required to improve data accessibility and operational responsiveness.



2.2 - System Architecture

- Designed a modular pipeline integrating image preprocessing, ROI detection, OCR-based data extraction, and structured data output.
- Enabled batch processing of thermal images and automated conversion into engineering datasets.
- Architecture supported extensibility for future AI model integration and edge deployment scenarios.

Image Processing in Four Steps

- Tradition Digital Image Process
  - template comparison
- Tradition Machine Learning
  - SVM, SVDD, PCA
- Python + OCR
  - EasyOCR
- Deep Learning

Tradition Digital Image Process

- Edge Detection（Sobel, Canny）
- Filtering Techniques（Gaussian, Median）
- Thresholding
- Morphological Operations（Dilation, Erosion）
- Handcrafted Features（SIFT, HOG, SURF）
- **Image Formation & Processing Fundamentals**
  - Illumination
  - Image Quality
  - Morphology
  - Discrete Cosine Transform (DCT)
  - Face Detection
  - Fourier Transform

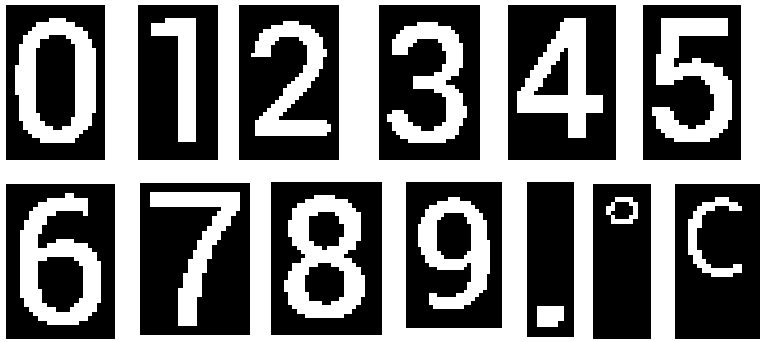
- **Image Enhancement & Restoration**
  - Smoothing Filters
  - Median Filtering
  - Weighted Averaging
  - Frequency Domain Filter Design

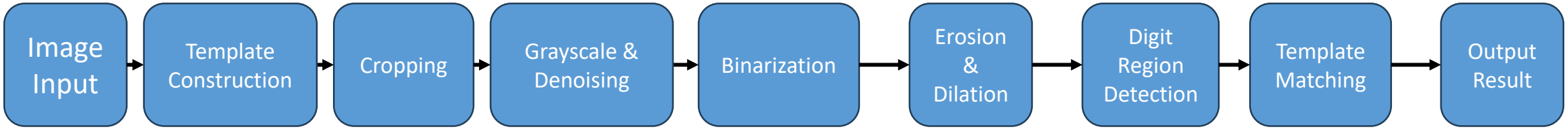
- Edge & Structural Analysis
  - Laplacian Operator
  - Basis Projection Methods
  - Contour Tracing
- Threshold Selection & Region Segmentation
- Classification
- Pattern Recognition
- Color Space Representation

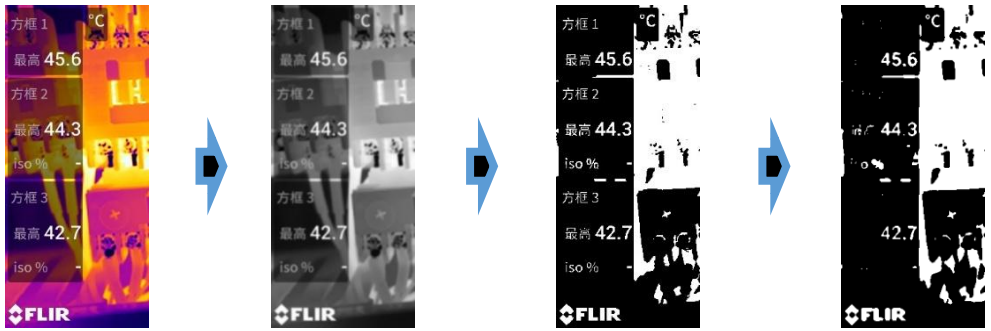
Image Processing Pipeline — Template Matching Approach

- **Processing Workflow**
- **Image Loading**
  Filter and load all images with filenames starting with **"IR"** and ending with **".JPG"**
- **Template Construction**
- **Image Preprocessing**
  - Crop the **left one-third region** (retain temperature digit area only)
  - Grayscale conversion and noise reduction
  - Binarization
  - Morphological operations (Erosion, Dilation)
- **Automatic Digit Region Detection**
- **Template Matching**

Tempplate Matching







Tradition Machine Learning

Supervised Learning

- **Classification Algorithms**
  - Decision Tree
  - Random Forest
  - Support Vector Machine (SVM)
  - k-Nearest Neighbors (K-NN)
  - Logistic Regression
  - Naive Bayes
  - Perceptron
  - Multi-Layer Perceptron (MLP)
- **Regression Algorithms**
  - Linear Regression
  - Ridge Regression
  - Lasso Regression
  - Support Vector Regression (SVR)
  - Regression Tree
  - Gradient Boosting Regression (e.g., XGBoost, LightGBM)

Unsupervised Learning

- **Clustering Algorithms**
  - DBSCAN
  - Mean Shift
  - Hierarchical Clustering
- **Dimensionality Reduction**
  - Principal Component Analysis (PCA)
  - t-SNE
  - UMAP
  - Independent Component Analysis (ICA)

Tradition Machine Learning— Key Methods

Support Vector Machine (SVM)

SVM is a supervised learning algorithm used for classification. Its objective is to find an optimal decision boundary (hyperplane) that separates different classes while maximizing the margin between them.
For non-linearly separable data, SVM applies the **kernel trick** to project data into a higher-dimensional space, enabling linear separation.

Support Vector Data Description (SVDD)

SVDD is a one-class classification method used for anomaly detection. It constructs a boundary around “normal” data by finding the smallest enclosing hypersphere that contains most training samples.
Data points outside this boundary are considered anomalies.

Principal Component Analysis (PCA)

PCA is an unsupervised dimensionality reduction technique that identifies directions (principal components) with the highest variance in the data.
It projects high-dimensional data into a lower-dimensional space while preserving as much information as possible.
Common applications include noise reduction, data visualization, and feature compression.

Independent Component Analysis (ICA)

ICA is used to separate statistically independent source signals from mixed observations.
A classic example is the **cocktail party problem**, where multiple microphones capture overlapping voices, and the goal is to separate each individual speaker.
Unlike PCA, ICA focuses on statistical independence (higher-order statistics), rather than mere uncorrelatedness.

Python + OCR

- Tesseract
  - A hybrid OCR system combining traditional computer vision techniques with **LSTM-based neural networks**
- EasyOCR
  - A deep learning-based OCR system built on **PyTorch**
  - Core architecture integrates **CNN + RNN + CTC Loss / Attention mechanisms**
- Pytesseract
  - Python wrapper for Tesseract OCR engine
- cv2(OpenCV)
  - Used for image preprocessing and computer vision operations
- Openpyxl
  - Used for writing and managing structured data in Excel files

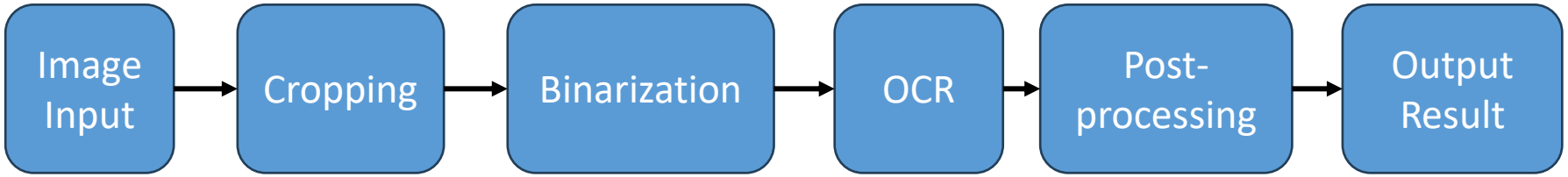
Image Processing Pipeline (OCR-Based Approach)

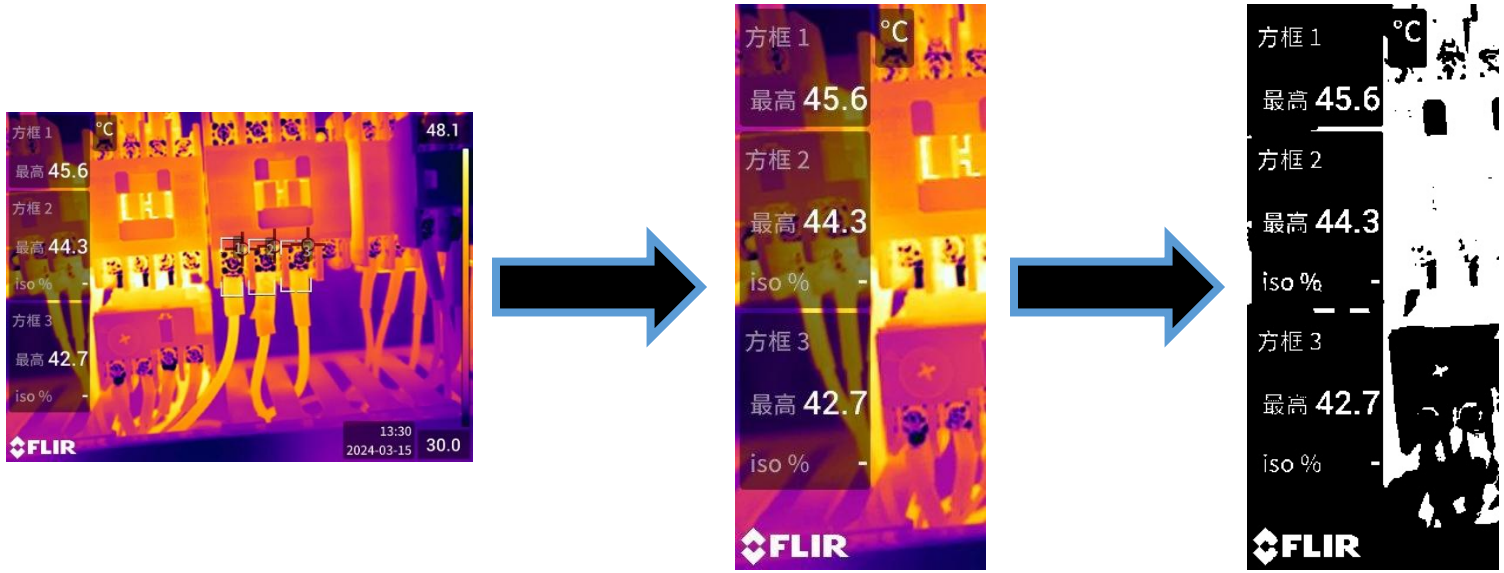
Processing Steps：

Image Loading: Filter and load all images with filenames starting with **"IR"** and ending with **".JPG"**

Image Preprocessing: Crop the **left one-third region** (retain only the temperature digit area)

Image Binarization: Through empirical comparison, **binarization provides better performance** than grayscale or denoising for digit extraction





Tesseract + Denoised Images

	oem 1 psm 7	oem 1 psm 8	oem 1 psm 11	oem 3 psm 6	oem 3 psm 7
Recognition Success	0%	0%	48%	19%	0%
Recognition Failure	0%	0%	29%	29%	0%
Unrecognized	100%	100%	23%	53%	100%

Tesseract + Binarized Images

	oem 1 psm 7	oem 1 psm 8	oem 1 psm 11	oem 3 psm 6	oem 3 psm 7
Recognition Success			55%		
Recognition Failure			38%		
Unrecognized			7%		

EasyOCR + Binarized Images + Post-processing

	oem 1 psm 7	oem 1 psm 8	oem 1 psm 11	oem 3 psm 6	oem 3 psm 7
Recognition Success			97%		
Recognition Failure			2%		
Unrecognized			1%		

Conclusion:
This study improved the recognition accuracy from **55% to 97%**, representing an increase of **42 percentage points**, while reducing the error rate by approximately **6x**.

Deep Learning

Objective

Design and train a model from scratch to achieve: **Image Input → Real-time Recognition → Temperature Value Output to Excel**

Technical Details

Dataset Collection & Annotation

- Utilized **800 thermal images** collected from previous stages
- Manually annotated temperature regions with **bounding boxes** using **LabelImg**
- Exported annotations in **YOLO format** or **COCO JSON format**

Data Preprocessing

- Resized images to a fixed resolution (e.g., **320 × 320**)
- Normalized pixel values to **[0, 1]**
- Applied data augmentation:
  - Rotation
  - Brightness adjustment
  - Contrast adjustment

Model Architecture

Adopted a **two-stage design**:

Stage 1 — Temperature Region Detection

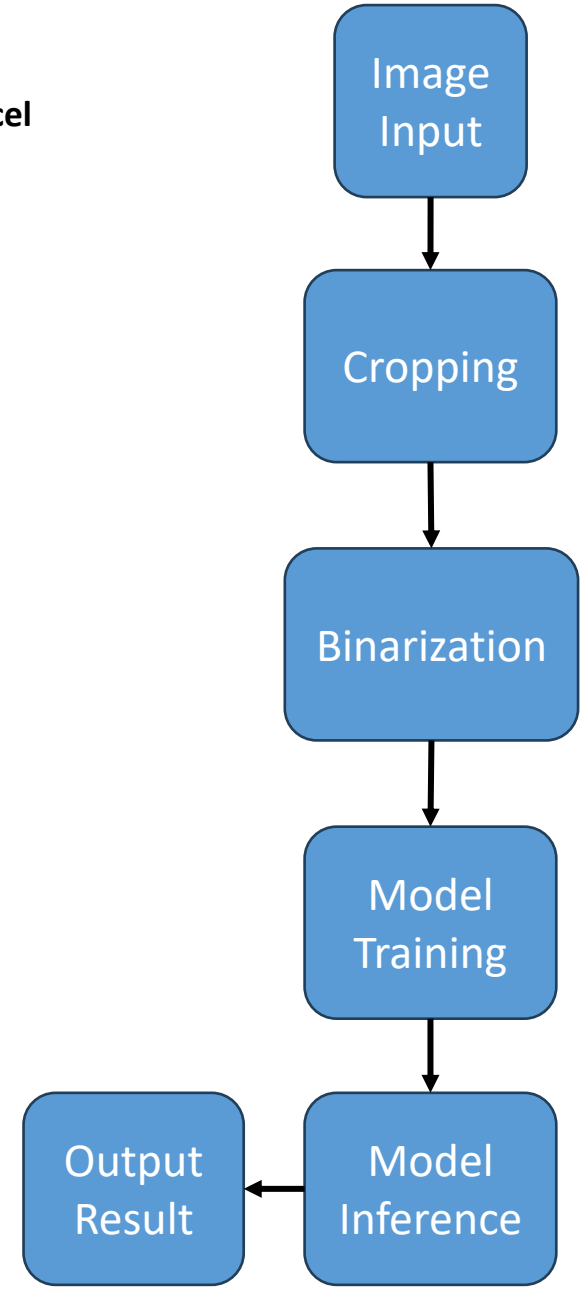
- Model: **YOLOv5s (lightweight version)**
- Output: Bounding box coordinates

Stage 2 — Digit Recognition

- Model: Custom **CNN-based model** (e.g., **LeNet-5** or modified **ResNet18**)
- **Input:** Cropped temperature region from detection output
- **Output:** Classification results for digits (0–9), decimal point, and ˚C

Future Extensions

- **Multilingual OCR** (Chinese, English, Japanese, Korean)
- **Scene Text Recognition** (natural environments, e.g., street signs)
- **Real-time Deployment** (e.g., Jetson Nano / Raspberry Pi)



2.3 - Model Evolution

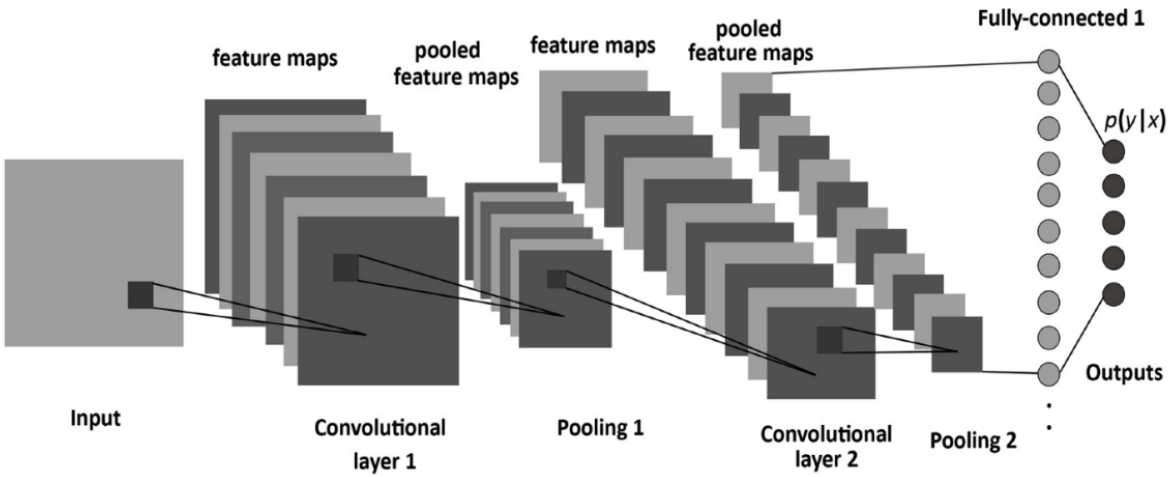
- Initial implementation utilized **traditional image processing and OCR techniques** for data extraction.
- Evaluated **machine learning and object detection models** to improve robustness under varying environmental conditions.
- Continuously refined recognition strategies based on **real production feedback** and **dataset characteristics**.

YOLOv5s — Text Region Detection

- **Advantage:** Fast inference speed and strong robustness against **lighting variations** and **noise**

CNN-Based Digit Recognition Model — Improved LeNet

- Designed a customized **CNN model** based on **LeNet architecture** for digit recognition



2.4 - Deployment Impact

- Significantly reduced manual workload and improved data processing speed.
- Enabled faster anomaly identification and engineering response planning.
- Provided a scalable framework for future industrial AI deployment.

Hardware Setup

1. Main Workstation
GIGABYTE A7 K1 (RTX 3060 130W)

- Primarily used for **ML/AI training tasks** (YOLO, Transformer, CNN)
- Equipped with a dedicated GPU, enabling **CUDA / PyTorch acceleration**

2. Secondary Laptop
Acer Swift Go 14 (Intel Core Ultra 9 285H)

- **High CPU Multi-core Performance**
  Suitable for daily development (**Python / C# / VS Code / multi-task compilation**)
- **32GB RAM + 1TB SSD**
  Ensures smooth performance when running IDEs, local demos, or handling moderately large datasets
- **Portable & Lightweight**
  More portable than the A7 K1, suitable for **coding, presentations, meetings, and demo showcasing**
- **Local AI Inference / Small Model Testing**
  Even without a dedicated RTX GPU, supports inference using **Intel NPU + CPU**, capable of running frameworks such as **ONNX Runtime** and **OpenVINO**

Part 3 - Industrial AI System Engineering Projects

- This portfolio presents selected industrial AI engineering projects that demonstrate my approach to problem framing, system architecture design, rapid validation, and real-world deployment planning.
- Each project highlights how AI technologies, automation workflows, and system integration strategies were applied to improve operational efficiency, data visibility, and engineering decision-making in manufacturing environments.

NVIDIA Jetson series

Jetson Thor 系列

最高 2070 TOPS
40–130 W
100 mm x 87 mm

即將上市

Jetson AGX Orin 系列

最高 275 TOPS
15–60W
100mm x 87mm
售價 899 美元起

Jetson Orin NX 系列

最高 100 TOPS
10–25W
70mm x 45mm
售價 399 美元起

Jetson Orin Nano™ 系列

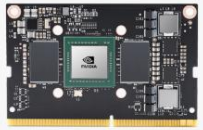
最高 40 TOPS
7-15W
70mm x 45mm
售價 199 美元起

Jetson AGX Xavier™ 系列

最高 32 TOPS
10-30W | 20-40W
100mm x 87mm
售價 899 美元起

Jetson Xavier NX 系列

21 TOPS
10-20W
70mm x 45mm
售價 399 美元起



Jetson TX2 系列

1.3 TFLOPS
7.5-15W | 10-20W
最小尺寸 70mm x 45mm
售價 149 美元起

Jetson Nano™

0.5 TFLOPS
5-10W
70mm x 45mm
99 美元

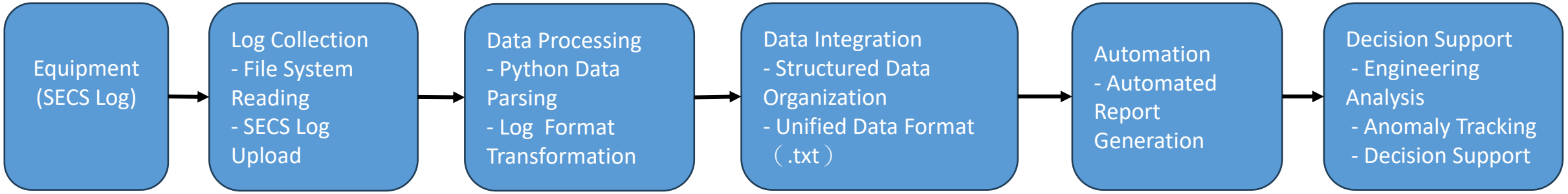
NVIDIA.

Part 3 - Workflow Issue

- Production data was distributed across multiple systems and logs, requiring manual consolidation.
- Lack of automation reduced data timeliness and limited decision-making effectiveness.
- Engineering teams faced operational delays due to inefficient information flow.

3.2 - Integration Architecture

- Designed automation workflows integrating SECS logs, file systems, and reporting processes.
- Built structured data pipelines enabling continuous data aggregation and monitoring.
- Architecture emphasized reliability, maintainability, and minimal disruption to existing operations.



3.3 - Automation Implementation

- Implemented Python scripts, RPA workflows, and scheduling mechanisms to automate repetitive tasks.
- Integrated office data processes with engineering data pipelines.
- Improved operational consistency and reduced manual intervention.

3.4 - Impact

- Establish automated equipment log upload and data integration processes to transform scattered, manually processed data into centralized and automated data, enhancing production line monitoring and problem response.

- This transforms data acquisition and analysis time from manual processing (days) to systematic processing (minutes), significantly improving engineering response speed.

- Build a scalable data infrastructure to support anomaly tracking and AI application development.

- Provide real-time data support to equipment and manufacturing process engineers, enhancing the efficiency of subsequent manufacturing process analysis and decision-making, and improving problem-solving capabilities.

Part 4 - Dataset Preparation Challenge in Real Factory Environment

- Real-world safety scenarios involved diverse visual conditions, including occlusion, lighting variation, and inconsistent object appearance.
- Initial dataset collection required manual frame extraction and labeling strategy definition to ensure meaningful model training.
- Multiple object categories and ambiguous boundaries increased annotation complexity and affected early detection performance.
- Iterative dataset refinement was necessary to improve model robustness and detection reliability.

4.1 - Model Training Validation and Iteration

- YOLO-based object detection models were evaluated for feasibility in safety monitoring scenarios.
- Early training results revealed class imbalance and insufficient feature representation in certain conditions.
- Hyperparameter tuning and dataset restructuring were applied to improve detection accuracy.
- Validation focused on practical deployment feasibility rather than benchmark-only performance metrics.

4.2 - Handling Real Deployment Constraints

- Considered camera positioning limitations, network bandwidth constraints, and processing latency requirements.
- Evaluated trade-offs between detection accuracy and inference speed for edge deployment scenarios.
- Explored integration approaches with existing automation systems and safety workflows.

4.3 - Safety Detection System Deployment Concept

- Proposed an edge AI-based safety monitoring framework combining camera input, detection models, and alert mechanisms.
- Designed conceptual data flow for real-time event detection and engineering response support.
- Identified future scalability opportunities for centralized monitoring and predictive safety analytics.

Q 1 | Objective

A1：Validate whether **AI can be applied to warehouse safety anomaly detection**

Q 2 | Why Did It Keep Failing?

- Annotation Tool (**LabelImg**)
- Data Structure (**YOLO Dataset**)
- Training Configuration (**data.yaml**)
- Inference Validation (**predict / infer**)

Data structure and annotation correctness are more critical than the model itself

Q 3 | How Was It Fixed?

- Discovered that the model was not learning (**Instances = 0**)
- Re-examined dataset and annotations
- Identified issues: too many classes and insufficient data
- Reduced problem scope for initial validation
- Retrained the model and achieved successful inference

Q 4 | Current Results (Beyond Just Numbers)

- **Recall:** Acceptable → anomalies can be detected
- **Precision:** Low → high false positive rate
- Conclusion is **not** "the model is poor"
- Instead:
  - Under current data volume, inconsistencies in annotations significantly affect performance

Q 5 | How to Improve?

1. Short-term (1–2 months)

- Increase dataset size
- Stabilize model performance
- Reduce false positives

2. Mid-term

- Replace baseline with **geometric / rule-based methods**
- Let YOLO focus on **object detection tasks**

3. Long-term

- Real-time monitoring
- Integration with production line systems
- Deployment as a **safety module**

1. Data Collection
– Collected **30 warehouse rail images** (normal / abnormal)

2. **Data Labeling**
– Annotated using **LabelImg / Roboflow**:
- **Person**
- **Ladder**
- **Tray**: Black / Transparent (**Tray_Black / Transparent_Tray**)
- **Object Drop**
- **Baseline**: **Baseline_Left / Baseline_Right**

– Generated annotations in **YOLO format (.txt)**

3. **Model Training**
– Trained models using **YOLOv8 / YOLOv9 / YOLOv11**
– Applied **train / validation split**
– Generated **best.pt** (trained object detection model)

4. **Inference / Detection**
– Input: **single image / video / real-time camera**
– YOLO model outputs:
- **Bounding boxes + class labels**

– Computation:
- Detect presence of **dropped objects**
- Check whether **tray protrusion** occurs
- **Object count statistics**
- **Person detection / Ladder detection**

5. **Visualization UI / Reporting**
– Display **bounding box overlay**
– Display **classification results, labels, confidence scores**
– Display **statistical information**:
- Object count
- Classification results
- Protrusion alerts
- Drop alerts
- Performance metrics (**Accuracy / Precision / Recall / mAP**)

– Can be implemented as:
- **Desktop application / Web UI / Dashboard**

Data Collection → Data Labeling → Model Training → Inference → UI Visualization

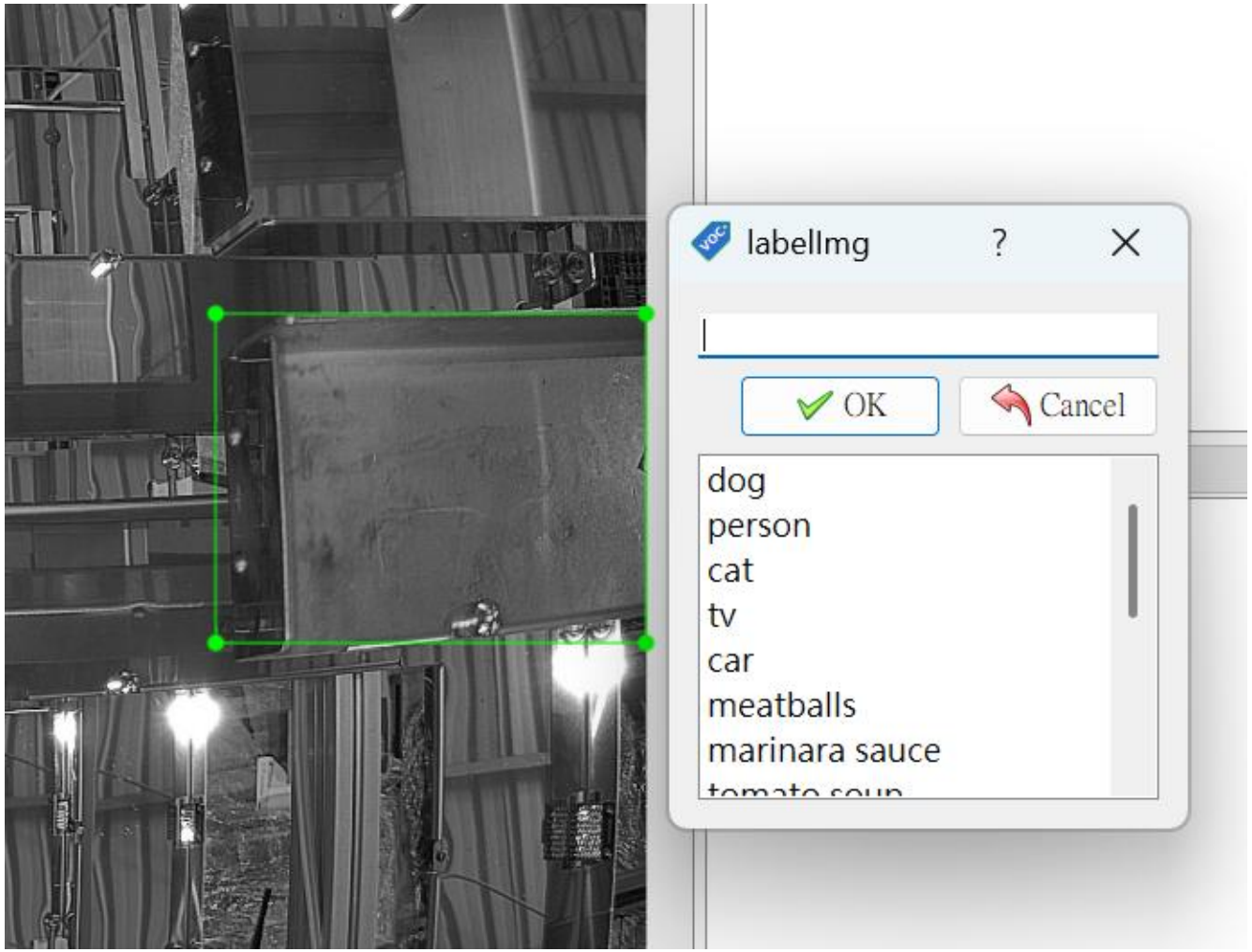
1. Data Collection
– Collected **30 warehouse rail images** (normal / abnormal)

2. **Data Labeling**

– Annotated using **LabelImg / Roboflow**:

- **Person**
- **Ladder**
- **Tray**: Black / Transparent (**Tray_Black / Transparent_Tray**)
- **Object Drop**
- **Baseline**: **Baseline_Left / Baseline_Right**

– Generated annotations in **YOLO format (.txt)**



YOLO Annotation Rule

The **class index in the .txt file must be within the range 0 to (nc - 1)**

```
0 0.740741 0.834105 0.216049 0.279321
0 0.723573 0.538066 0.205633 0.299383
0 0.614583 0.346965 0.038580 0.233025
0 0.600309 0.420267 0.038580 0.204733
```

6 Classes：
0.person
1.ladder
2.tray_black
3.transparent_tray
4.object_drop
5.baseline

Q 1：Class Index Misalignment in LabelImg

- LabelImg uses a built-in **classes.txt** file
- Misalignment in class index ordering can lead to incorrect annotations

Q 2：Instances Must Be > 0

Observed in training logs:

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       2/20         0G          0      31.12          0          0        640: 100% ━━━━━━━━━━━━ 5/5 2.2it/s 2.3s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 3.7it/s 0.5s
C:\Users\User\AppData\Local\Programs\Python\Python310\lib\site-packages\ultralytics\utils\metrics.py:832: RuntimeWarning: Mean of empty slice.
  i = smooth(f1_curve.mean(0), 0.1).argmax()  # max F1 index
C:\Users\User\AppData\Local\Programs\Python\Python310\lib\site-packages\numpy\_core\_methods.py:137: RuntimeWarning: invalid value encountered in di
vide
  ret = um.true_divide(
                   all         10          0          0          0          0          0
WARNING no labels found in detect set, can not compute metrics without labels
```

Instances = 0 indicates:

- YOLO did not read any **valid labeled bounding boxes** in the current batch
- The model effectively **has nothing to learn**

Debugging & Validation Process

1. **Re-evaluated the entire workflow** to verify whether the **AI pipeline is feasible end-to-end**
2. **Reduced the number of classes from 6 to 2** to simplify the problem for initial validation
3. **Corrected dataset directory structure** to comply with YOLO requirements
   - image → images
4. **Developed a script** to modify all .txt annotation files
   - Ensured class indices are correctly mapped (e.g., starting from 0 and 1)
5. **Final validation**
   - Verified whether the model **converges during training**

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
14/20         0G      1.377      4.135      1.388          5        640: 100% ━━━━━━━━━━━━ 5/5 2.2it/s 2.2s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 3.1it/s 0.6s
             all         10         16      0.003      0.731     0.0353     0.0173

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
15/20         0G      1.165      4.584      1.111          5        640: 100% ━━━━━━━━━━━━ 5/5 2.2it/s 2.2s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 3.4it/s 0.6s
             all         10         16     0.0033      0.769     0.0306     0.0177

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
16/20         0G     0.8727      8.984     0.9353          2        640: 100% ━━━━━━━━━━━━ 5/5 2.3it/s 2.1s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 3.2it/s 0.6s
             all         10         16     0.0033      0.769     0.0306     0.0177

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
17/20         0G     0.7202      9.028     0.8723          6        640: 100% ━━━━━━━━━━━━ 5/5 2.2it/s 2.2s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 3.1it/s 0.7s
             all         10         16     0.0033      0.769     0.0306     0.0177
```

Training Results Analysis

•**epochs = 20**

→ Number of training epochs

•**imgsz = 640**

→ All input images are resized to **640 × 640**, regardless of original resolution, to ensure a consistent input size

•**batch = 4**

→

- **batch = 1**: processes one image at a time (slower but more stable)
- **larger batch size**: faster training but higher memory consumption
- Current setup: model processes **4 images per iteration** before updating weights

•**device = CPU**

mAP50 ≈ 0.108

→ Represents the model's **overall detection capability**

→ Range: **0 to 1**, higher is better

→ A detection is considered correct when **IoU ≥ 50%**

•**Precision:** Low

•**Recall:** Moderate (acceptable)

Conclusion

These results indicate that the model has **successfully learned meaningful features** from the data.

Part 5 - Future Concept

- Envisioned a centralized platform integrating equipment data, vision-based AI insights, and operational analytics.
- The system would enable real-time monitoring, anomaly detection, and decision support across multiple production lines.
- Designed as a scalable architecture supporting progressive AI adoption in manufacturing environments.

5.2 - Edge AI Architecture Roadmap for Industrial Deployment

- This paper proposes an AI deployment strategy centered on edge computing devices to reduce inference latency and improve immediacy.
- It constructs an integrated architecture of edge AI, factory networks, and a central monitoring platform to achieve real-time data feedback and decision support.
- It implements digital twin and simulation verification mechanisms, creating a virtual environment using NVIDIA Isaac Sim and Omniverse.
- It develops Sim-to-Real (virtual-physical integration) capabilities to shorten development and deployment cycles and reduce on-site trial-and-error costs.